Reward Design for RL-Based Token-Level Prompt Compression

Anonymous Authors[1]

Abstract

Large language model inference cost scales linearly with prompt length, yet prompts routinely contain redundant tokens. Prior work compresses prompts using information-entropy heuristics or supervised classifiers, but these approaches are task-agnostic or require expensive distillation labels. Recent RL-based methods (PCRL, TACO-RL) learn compression policies from task feedback, but use purely sparse terminal rewards, limiting training signal. We study reward design for token-level prompt compression, comparing REINFORCE and MCTS across four reward functions (gated, continuous, soft-gated, harmonic) with a frozen DistilRoBERTa encoder and lightweight trainable head. We identify and resolve two reward pathologies—a *keep/drop asymmetry* in per-token KL rewards that drives drop-all collapse, and a *faithfulness mismatch* where comparing against ground-truth answers produces near-zero reward signal. A $2\times2\times2$ factorial over algorithm (REINFORCE, MCTS), reward (harmonic, gated), and learning rate reveals the reward function as the primary lever for the compression–faithfulness tradeoff: harmonic reward achieves up to 74% compression while gated reward preserves 95% of uncompressed task F1 at 43% compression. The best configuration (MCTS+gated, lr=3×10^{-4}) achieves $3.5\times$ improvement over LLMLingua-2 on SQuAD 2.0 extractive QA.

1. Introduction

Large language models (LLMs) achieve strong performance across diverse NLP tasks, but their inference cost scales linearly with input sequence length. In many practical settings—retrieval-augmented generation, document QA, multi-turn dialogue—prompts contain substantial redundancy. *Prompt compression* aims to reduce token count while preserving task-relevant information, thereby decreasing latency and cost.

Prior approaches fall into two categories. *Heuristic methods* such as Selective Context (**?**) and LLMLingua (**?**) use information-theoretic criteria (self-information, perplexity) to score and prune tokens. While effective, these methods are task-agnostic and may discard tokens critical for specific downstream tasks. *Supervised methods* like LLMLingua-2 (**?**) train token classifiers on distillation labels from GPT-4, achieving strong performance but requiring expensive label generation.

Recent work has explored *reinforcement learning* for prompt compression. PCRL (**?**) uses REINFORCE with a self-critical baseline and a sparse faithfulness reward (ROUGE-L between original and compressed LLM outputs). TACO-RL (**?**) applies REINFORCE with task-specific metrics (F1 for QA, BLEU for summarization) and a compression-rate tolerance. Both methods use purely sparse terminal rewards, distributing a single scalar to all token decisions equally.

A natural extension is to add *dense* reward shaping via KL divergence between the LLM's output distributions on the full and compressed prompts, providing per-token gradient signal rather than a single episode-level scalar. However, combining dense and sparse rewards introduces subtle pathologies that prior work has not addressed.

We present a focused study of reward design for RL-based prompt compression, making three contributions:

1. We identify a **keep/drop reward asymmetry** in per-token KL rewards: kept tokens receive negative reward (KL ≥ 0) while dropped tokens receive zero, creating an incentive to drop all tokens regardless of their importance. We resolve this by distributing the total KL signal equally across all tokens.

2. We show that comparing compressed-prompt LLM output against **ground-truth answers** (as in standard QA evaluation) produces near-zero reward signal at moderate compression ratios, because a compressed prompt rarely yields the exact answer. Following PCRL and TACO-RL, we instead use a **faithfulness objective**:

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country.**AUTHORERR: Missing \icmlcorrespondingauthor.**

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

F1($\mathcal{M}(\tilde{\mathbf{x}}), \mathcal{M}(\mathbf{x})$), comparing the LLM's behavior on compressed vs. original input.

3. A **diagnostic sweep** over 7 configurations (3 algorithms, 3 reward types, 2 architectures) confirms that reward design dominates algorithm and architecture choice. The contextual bandit with hybrid reward is the only configuration achieving nonzero task reward; pretrained encoders collapse under RL gradients.

We evaluate on extractive QA (SQuAD 2.0 (**?**)) with a frozen Llama-3.1-8B, comparing against random dropping, Selective Context, and LLMLingua-2 baselines.

2. Related Work

Heuristic prompt compression. Selective Context (**?**) computes per-token self-information using a small language model and retains the most informative tokens. LLMLingua (**?**) extends this with a budget controller and iterative compression, achieving 2–5× compression with minimal performance loss. These methods require no training but are task-agnostic.

Supervised prompt compression. LLMLingua-2 (**?**) trains a DistilBERT token classifier on labels distilled from GPT-4, framing compression as binary token classification. This achieves state-of-the-art results but requires expensive label generation and may not generalize across tasks without re-distillation.

RL-based prompt compression. PCRL (**?**) uses REINFORCE with a self-critical baseline to learn a compression policy. Crucially, PCRL measures *faithfulness*—ROUGE-L between $\mathcal{M}(\tilde{\mathbf{x}})$ and $\mathcal{M}(\mathbf{x})$—rather than comparing against ground-truth labels. The reward is sparse and binary: $1 - \rho$ if faithfulness exceeds a threshold $\tau = 0.9$, else -0.01. TACO-RL (**?**) similarly compares compressed and original LLM outputs, using task-specific metrics (F1 for QA, BLEU for summarization) with a compression-rate tolerance band. Both methods use purely sparse terminal rewards with REINFORCE, leaving open whether dense reward shaping can improve credit assignment.

Reward shaping in RL. Dense reward signals can accelerate RL training by providing more frequent feedback (**?**). In prompt compression, KL-divergence between full-prompt and compressed-prompt output distributions provides a natural dense signal. However, naive per-token KL rewards introduce pathologies that we identify and resolve in this work.

3. Methods

3.1. Problem Formulation

Given a prompt $\mathbf{x} = (x_1, \ldots, x_N)$ of N tokens and a frozen LLM $\mathcal{M}$, the compression agent processes tokens sequentially and emits binary actions $a_i \in \{0, 1\}$ (drop/keep). The compressed prompt $\tilde{\mathbf{x}} = (x_i : a_i = 1)$ is fed to $\mathcal{M}$, producing output $y = \mathcal{M}(\tilde{\mathbf{x}})$. The compression ratio is $\rho = |\tilde{\mathbf{x}}|/N$.

We process prompts in overlapping chunks of size C (default 128) with overlap O (default 16). The policy sees one chunk at a time, processing left-to-right with no look-ahead across chunks. Within a chunk, the policy architecture determines whether attention is bidirectional (DistilBERT) or causal (TinyLlama, custom Transformer with `causal=True`).

Episode structure. Each prompt forms one episode in a gym-style compression environment. The environment splits the prompt into overlapping chunks and presents them sequentially: `reset()` returns the first chunk as an observation (token IDs, attention mask, absolute position IDs, current compression ratio, and target ratio ρ^*), and `step(`a`)` accepts binary keep/drop actions for the current chunk and returns the next. Overlap tokens between consecutive chunks are resolved by an "earlier chunk wins" rule: the first O tokens of each chunk after the first are skipped during merge, so committed decisions are final. The environment tracks the running compression ratio incrementally and exposes it in each observation, enabling the policy to adjust its aggressiveness as the episode progresses. After the final chunk, the full keep mask is assembled and the compressed prompt is constructed via a *text bridge*: kept policy tokens are decoded back to text and re-tokenized with the LLM's tokenizer, producing a clean token sequence in the LLM's vocabulary without positional artifacts. This decode–re-tokenize step is essential in the dual-tokenizer setting, where the policy and LLM use different BPE vocabularies and a one-to-one token mapping does not exist. The terminal reward is then computed from the LLM's output on the re-tokenized compressed prompt.

3.2. Policy Architecture

The policy maps token features to per-token keep/drop logits $\pi_\theta(a_i|\mathbf{x}_{\text{chunk}}) \in \mathbb{R}^2$.

Custom Transformer. A lightweight Transformer trained from scratch (4 layers, 256 hidden dim, 4 heads, ∼2.7M parameters). We found that pretrained encoders (DistilBERT (**?**)) collapse catastrophically under RL gradients (entropy→0 within 100 episodes; see **??**), while a randomly-initialized network has no prior to fight against and remains stable across all reward types. The policy supports both causal and bidirectional attention modes.

DistilRoBERTa policy with dual tokenizer. To leverage pretrained representations without the DistilBERT collapse, we use DistilRoBERTa-base (82M parameters) with a trainable classification head. The head uses Xavier normal initialization (gain=0.1) on the output weight and bias $[0, 0.2]$ for a mild keep prior ($\sim$55% keep probability). Since DistilRoBERTa uses a different tokenizer than the frozen LLM (RoBERTa BPE vs. Llama BPE), we employ a dual-tokenizer architecture: the policy operates on RoBERTa tokens natively, and compression is realized through the text bridge described above—kept policy tokens are decoded back to text and re-tokenized with the LLM's tokenizer. This avoids forcing the policy to operate on a foreign vocabulary while producing clean LLM input without positional artifacts.

Ratio conditioning. An optional adapter that maps the target compression ratio $\rho^* \in [0, 1]$ to a hidden-dimension vector via a 2-layer MLP with GELU activation, fused with token features by addition. This enables a single policy to produce different compression ratios at inference time.

3.3. Reward Functions

Reward design is the central focus of this work. We identify two pathologies in naive reward formulations and present corrected versions. The final hybrid reward combines a dense KL-divergence signal with a sparse faithfulness terminal reward.

Sparse faithfulness reward. Following PCRL (**?**) and TACO-RL (**?**), the terminal reward measures *faithfulness*: how well the LLM's behavior is preserved after compression. Concretely, we generate outputs from both the original and compressed prompts and compute token-level F1 between them:

$$r_{\text{faith}} = \text{F1}(\mathcal{M}(\tilde{\mathbf{x}}),\ \mathcal{M}(\mathbf{x})) \cdot (1 - \rho) \tag{1}$$

The multiplicative $(1 - \rho)$ term balances quality and compression: at ρ=1.0 (no compression) the reward is zero regardless of faithfulness, preventing the keep-all degenerate solution. We also implement PCRL's threshold mode ($1 - \rho$ if faithfulness $\geq \tau$, else $-p$) and an additive mode (F1 $- \lambda|\rho - \rho^*|$) for comparison.

Why faithfulness, not accuracy. An initial design compared the compressed-prompt LLM output against *ground-truth answers* (standard QA evaluation). This produced near-zero reward: at $\rho \approx 0.5$, the LLM rarely generates the exact SQuAD answer from a compressed prompt (F1 $\approx$ 0.02), so the multiplicative reward provides essentially no gradient signal. Faithfulness is a fundamentally easier target—measuring whether compression *preserves behavior* rather than whether the LLM *answers correctly*—and produces 5–10$\times$ stronger reward signal (F1 $\approx$ 0.1–0.3 at moderate compression).

Dense KL reward with shared distribution. We augment the sparse faithfulness signal with a dense per-episode reward based on KL divergence between the LLM's output distributions on the full and compressed prompts:

$$r_{\text{KL}} = -\alpha \sum_{i \in \text{kept}} \text{KL}\Big[p_{\mathcal{M}}^{(i)}(\cdot|\mathbf{x}) \parallel p_{\mathcal{M}}^{(i)}(\cdot|\tilde{\mathbf{x}})\Big] \tag{2}$$

Full-prompt logits are cached once per episode since $\mathcal{M}$ is frozen, so only one additional forward pass (on the compressed prompt) is needed. We use forward KL, which penalizes the compressed distribution for failing to cover the support of the full-prompt distribution.

Keep/drop reward asymmetry (Pathology 1). A naive implementation assigns per-token KL reward only to *kept* tokens (since KL is defined only at positions present in both sequences), while *dropped* tokens receive zero reward. Since KL $\geq$ 0, kept tokens always receive non-positive reward $-\alpha \cdot \text{KL}_i \leq 0$. The resulting incentive structure is:

Action	Reward
Keep token	≤ 0 (negative KL penalty)
Drop token	$= 0$ (no signal)

The policy can only improve by dropping more tokens, leading to universal drop-all collapse within 1–2 episodes regardless of the token's importance. We resolve this by computing the total KL over kept positions and distributing it *equally to all tokens*:

$$r_i = \frac{r_{\text{KL}}}{N} \quad \forall\, i \in \{1, \ldots, N\} \tag{3}$$

Both kept and dropped tokens now share the consequence. The policy gradient naturally differentiates across episodes: when a token is kept and total KL decreases, the positive signal reinforces keeping; when it is dropped and KL increases, the negative signal discourages dropping. This removes the structural asymmetry while preserving the dense training signal that PCRL and TACO-RL lack.

Hybrid reward. The final reward combines shared dense KL (**??**) with sparse faithfulness (**??**):

$$r_{\text{hybrid}} = r_{\text{KL}}^{\text{shared}} + \mathbb{1}[\text{done}] \cdot r_{\text{faith}} \tag{4}$$

The terminal faithfulness reward is distributed only to kept tokens (following bug fix #14 from our codebase audit), since only keep decisions contribute to the compressed prompt's quality.

Teacher-forced cross-entropy (output-space dense reward). The input-space KL reward (**??**) requires aligned token positions between the original and compressed prompts, which the text bridge destroys. We propose an alternative dense signal in the *output* space. Given the full-prompt LLM output $y_{1:K} = \mathcal{M}(\mathbf{x})$, we measure how well the compressed-prompt model predicts these same tokens via teacher forcing:

$$r_{\text{CE}} = -\alpha \cdot \frac{1}{K} \sum_{i=1}^{K} \log P_{\mathcal{M}}(y_i \mid \tilde{\mathbf{x}}, y_{1:i-1}) \tag{5}$$

This requires one additional forward pass (not autoregressive generation) per episode. Unlike ROUGE-L, which treats all non-matching tokens identically, CE captures partial agreement in the full vocabulary distribution: a compressed model that assigns 40% probability to the correct next token receives more credit than one assigning 1%.

3.4. RL Algorithm

Contextual bandit. We treat each token's keep/drop decision as independent given a shared context encoding from the policy Transformer. There is no temporal credit assignment, discounting, or value function. The loss for a batch of episodes is:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log \pi_\theta(a_i|\mathbf{x}) \cdot r_i - \beta \mathcal{H}(\pi_\theta) \tag{6}$$

where r_i is the shared dense KL reward plus any terminal faithfulness reward distributed to token i, and $\beta = 0.05$ is the entropy bonus coefficient.

We also evaluated REINFORCE and DQN in our diagnostic sweep (**??**). REINFORCE with sparse reward showed no learning signal; with KL-dense reward it over-compressed ($\rho \to 0.06$). DQN with sparse reward also received zero terminal reward. The bandit's simplicity proved advantageous: independent per-token decisions with no sequential structure to add variance.

REINFORCE with SCST baseline. Following PCRL (**?**), we implement REINFORCE with a self-critical sequence training (SCST) baseline: k=4 sampled rollouts and one greedy baseline per prompt, with advantage $A = R_{\text{sample}} - R_{\text{greedy}}$ and gradient $-\sum_i \log \pi_\theta(a_i|\mathbf{x}) \cdot A$. We also support continuous reward mode where $r = \text{faithfulness}^2 \cdot (1 - \rho)$ replaces the gated threshold ($1 - \rho$ if faithfulness ≥ 0.9, else -0.01), providing non-zero gradient signal even when faithfulness is below the threshold. Token selection uses Gumbel-max top-k sampling; we approximate the Plackett-Luce log-probability via per-token log-softmax rather than independent Bernoulli, ensuring the policy gradient creates competition between tokens—increasing one token's score decreases others' selection probability.

AlphaZero-style policy pre-training. Inspired by AlphaZero's supervised warm-start before self-play (**?**), we bootstrap the policy head with per-token importance labels before MCTS training. For each of ∼200 SQuAD prompts, we generate K=20 compression masks (10 contiguous block drops at evenly-spaced positions + 10 random masks, all at the target keep rate) and measure faithfulness via the text bridge pipeline. We then compute per-token importance via marginal contribution: $\text{importance}(t_i) = \mathbb{E}[f \mid t_i \text{ kept}] - \mathbb{E}[f \mid t_i \text{ dropped}]$, where f is ROUGE-L faithfulness. Importance scores are normalized to $[0, 1]$ per prompt and the policy head is trained with MSE loss for 10 epochs. This gives MCTS a meaningful prior over which tokens to explore keeping vs. dropping, avoiding the initial random-drop phase that shatters BPE sub-words.

3.5. Non-RL Baselines

We compare against three non-RL baselines spanning the spectrum from task-agnostic random dropping to trained token classifiers.

Random drop. Each token is independently kept with probability equal to the target keep ratio, using Bernoulli sampling with a fixed seed for reproducibility. Serves as a lower bound; tokens are discarded without regard to content or position.

Selective Context. Scores each token by its self-information $-\log P(x_i \mid x_{<i})$ computed from the same frozen Llama-3.1-8B used for reward computation (unlike the original paper (**?**), which used a smaller LM). Tokens are grouped into noun-phrase units using spaCy's dependency parser; the score of a phrase is the mean self-information of its constituent tokens. Units are sorted descending by score and greedily selected until the target compression ratio is met. Special tokens (BOS/EOS) are always retained.

LLMLingua-2. Uses `microsoft/llmlingua-2-bert-base-mult` a BERT-based multilingual classifier that assigns per-token "keep" probabilities via binary token classification. The top keep-ratio fraction of tokens by keep probability are retained. Since BERT has a 512-token context limit, LLMLingua-2 is excluded from MeetingBank evaluation (transcripts average ∼4k tokens; BERT would see less than 15% of each document).

3.6. Experimental Setup

Frozen LLM. We use two LLM configurations. Our diagnostic sweep and baseline evaluations use Llama-3.1-8B (**?**) in 4-bit NF4 quantization. For REINFORCE training with the DistilRoBERTa policy, we switched to Llama-3.2-1B (also 4-bit quantized) due to computational constraints on consumer GPUs: the larger model's 2-second generation

time per episode made k=4 SCST training prohibitively slow at 2000 episodes. Both use bitsandbytes double quantization with bfloat16 compute dtype. For CPU-only testing, full-precision float32 weights are loaded without quantization.

Dataset. SQuAD 2.0 (**?**) (extractive QA). We report both *faithfulness F1* (reward signal: $\text{F1}(\mathcal{M}(\tilde{\mathbf{x}}), \mathcal{M}(\mathbf{x}))$) and *actual F1* (diagnostic: $\text{F1}(\mathcal{M}(\tilde{\mathbf{x}}), y^*)$ against ground-truth answers).

Baselines. Random token dropping, Selective Context (**?**), and LLMLingua-2 (**?**).

Evaluation. Compression ratio sweep at $\rho \in \{0.3, 0.4, 0.5, 0.6, 0.7\}$ on the best checkpoint. Three seeds per configuration.

Experiment tracking. All runs are logged to Weights & Biases, including per-episode KL reward, faithfulness reward, compression ratio, and entropy.

Table 1. Hyperparameters for the contextual bandit with hybrid reward.

Hyperparameter	Value
Learning rate	3×10^{-4}
Entropy coefficient β	0.05
Max gradient norm	1.0
Chunk size C	128
Chunk overlap O	16
KL coefficient α	0.001
Target compression ratio ρ^*	0.5
Max new tokens (LLM generation)	256

4. Results

4.1. Non-RL Baseline Results

?? reports proxy scores for the three non-RL baselines across both datasets and five compression ratios. Scores are proxies for downstream task quality (see **??**): for SQuAD we report token-level F1 between the compressed prompt and the gold answer span, and for MeetingBank we report ROUGE-L between the compressed transcript and the reference summary. Both metrics measure how much task-relevant information survives compression, not actual LLM generation quality; true evaluation requires a full LLM inference pass over the compressed prompt. Note that proxy F1 values are low (1–4%) because even moderate compression drops answer-span tokens; however, the LLM may still answer correctly from surrounding context, so these proxies are conservative lower bounds on actual task performance.

Table 2. Non-RL baseline proxy scores at five compression ratios. SQuAD: token-level F1 between compressed prompt and gold answer (%). MeetingBank: ROUGE-L between compressed transcript and reference summary (%). LLMLingua-2 is excluded from MeetingBank (BERT 512-token limit; transcripts avg. ∼4k tokens). Higher is better (↑). Results averaged over 200 validation examples.

Dataset	Method	ρ=0.3	ρ=0.4	ρ=0.5	ρ=0.6	ρ=
3*SQuAD	Random Drop	1.67	1.59	1.48	1.67	1.
	Selective Context	0.97	1.11	1.30	1.41	1.
	LLMLingua-2	3.73	3.07	2.58	2.31	2.
2*MeetingBank	Random Drop	13.49	13.33	12.70	12.81	13
	Selective Context	13.69	14.46	15.02	15.20	14

Three patterns emerge. First, LLMLingua-2 substantially outperforms both baselines on SQuAD at low compression ratios (3.73% vs. 1.67% at ρ=0.3), but its advantage shrinks at higher ratios—by ρ=0.7 all three methods converge. This is consistent with LLMLingua-2's token classifier being calibrated for aggressive compression. Second, Selective Context *underperforms* random drop on SQuAD at ρ=0.3–0.4. This is expected: Selective Context retains tokens with high self-information (surprising, low-probability tokens), but answer spans in reading-comprehension prompts tend to be *predictable in context* (low self-information), so they are selectively *dropped*. Third, on MeetingBank, Selective Context consistently outperforms random drop and improves monotonically with ρ, suggesting that high-self-information tokens in meeting transcripts are more informative for summarization than for extractive QA.

4.2. Preliminary RL Results: REINFORCE

We report preliminary results from our first RL training run: REINFORCE with the CustomTransformer policy and sparse (F1) reward on SQuAD, targeting ρ=0.5. The run completed 2,050 of 10,000 planned episodes before a software bug terminated it (an empty `answer_texts` edge case in the reward function, now fixed).

Policy collapse. We distinguish two degenerate failure modes, both characterized by entropy collapse (entropy $\to$ 0): *keep-all collapse* ($\rho \to 1.0$), where the policy learns to retain every token, and *drop-all collapse* ($\rho \to 0$), where the policy drops nearly everything. Within approximately 150 episodes, the policy exhibited keep-all collapse: compression ratio $\rho \to 1.0$, action entropy dropping from 0.52 to below 0.01. Mean episodic return stabilized at 0.023, dominated by the compression penalty (-0.005 per episode $= \lambda \cdot |1.0 - 0.5|$ with λ=0.01). We observed occasional positive reward spikes (0.05–0.25) when the downstream LLM answered correctly despite no compression, but these were too infrequent to counteract the penalty gradient. Policy loss converged to near zero as the gradient signal vanished

with entropy collapse.

Hypothesized causes. We attribute the collapse to two factors. First, the sparse reward provides a weak and noisy learning signal: the F1 score depends on the downstream LLM's generation, which introduces high variance unrelated to the compression policy's action. The policy receives meaningful positive signal only when the LLM happens to produce a correct answer, which is rare early in training when the policy has not yet learned useful compression patterns. Second, the compression penalty is continuous and deterministic, providing a consistent gradient toward ρ=1.0 (no compression), while the task reward is stochastic and episodic. The policy thus follows the path of least resistance: maximize reward by minimizing penalty, ignoring the noisy task signal.

Implications. These results motivate two immediate extensions: (1) dense reward shaping via KL divergence between compressed and original LLM logits, which provides per-token gradient signal independent of downstream generation quality; and (2) entropy regularization to prevent premature convergence. We explore these in subsequent experiments.

Reward landscape fix. We addressed the collapse with three changes validated against published RL compression work (**??**). First, we replaced the additive sparse reward with a *multiplicative* formulation (**??**): $r = \text{metric}(y, y^*) \cdot (1-\rho)$, which yields zero reward at ρ=1.0 regardless of task score, eliminating the keep-all optimum. Second, we set the discount factor γ=1.0 for all on-policy algorithms (REINFORCE, PPO, bandit). With sparse terminal reward over $\sim$1024 tokens, γ=0.99 attenuates early-token signals by $0.99^{1023} \approx 3 \times 10^{-5}$, effectively zeroing the gradient for most of the sequence. Both PCRL and TACO-RL use undiscounted returns. Third, we increased the entropy coefficient from 0.01 to 0.05, adding a bonus of up to $\sim$0.035 (at maximum binary entropy $\ln 2 \approx 0.69$) to prevent premature convergence to a deterministic policy. We also reduced the KL coefficient from 0.1 to 0.01 so that dense KL shaping provides directional signal without dominating the terminal task reward.

4.3. Diagnostic Sweep: Algorithm × Reward × Architecture

To validate the reward landscape fix and identify viable configurations for longer runs, we conducted a 100-episode diagnostic sweep across 7 configurations spanning three algorithms (REINFORCE, DQN, contextual bandit), three reward types (sparse, KL-dense, hybrid), and two architectures (CustomTransformer, DistilBERT). All runs target $\rho^* = 0.5$ on SQuAD. **??** summarizes the results.

Table 3. 100-episode diagnostic sweep on SQuAD ($\rho^* = 0.5$). Ratio: mean compression ratio achieved. Logged Rwd: mean episodic reward as logged by W&B. Entropy: mean action entropy (max binary entropy $= \ln 2 \approx 0.69$); DQN uses ε-greedy exploration rather than a stochastic policy, so entropy is not defined. CT = CustomTransformer, DB = DistilBERT. **Note:** Logged reward reflects different signals per reward type: for sparse configs, terminal F1 $\times$ $(1-\rho)$; for KL-dense, cumulative per-token KL penalty; for hybrid, the sparse terminal component only. Values are not directly comparable across reward types. The key comparison is whether any config achieved nonzero *task* reward (F1 signal).

Config	Ratio	Logged Rwd	Ent.	Key Finding
REINFORCE+CT+KL	0.059	−0.64	0.21	Over-compression
REINFORCE+CT+Hybrid	0.144	0.00	0.42	Zero terminal rewar
REINFORCE+CT+Sparse	0.502	0.00	0.69	No learning signal
Bandit+CT+Hybrid	**0.306**	**0.15**	**0.62**	**Only nonzero task**
Bandit+CT+KL	0.255	−2.25	0.58	Very negative rewar
REINFORCE+DB+KL	0.004	−0.13	0.00	Drop-all collapse
DQN+CT+Sparse	0.524	0.00	N/A	No reward signal

Three findings emerge. First, **dense reward is necessary**: both sparse-only configurations (REINFORCE+CT+Sparse, DQN+CT+Sparse) received zero terminal reward across all 100 episodes, confirming that at aggressive compression ratios the multiplicative F1 reward provides no gradient signal. Second, **Bandit+CT+Hybrid is the only viable configuration**: it alone achieved nonzero task reward (0.15), maintained a healthy compression ratio (0.306), and preserved high action entropy (0.62). Third, **DistilBERT exhibits drop-all collapse**: entropy dropped to 0.002 and ratio to 0.004 within 100 episodes, indicating that the pretrained bidirectional representations are too rigid for RL fine-tuning on this novel objective. The from-scratch CustomTransformer remained stable across all reward types.

Based on these results, we selected Bandit+CT+Hybrid as the first configuration for extended training.

4.4. Reward Pathology Analysis

Before presenting training results, we document two reward pathologies discovered during development that fundamentally shaped the final system design.

Pathology 1: Keep/drop asymmetry causes drop-all collapse. With naive per-token KL rewards assigned only to kept positions, the policy collapses to dropping all tokens within 1–2 episodes. **??** shows a representative run: episode 1 achieves $\rho = 0.57$ but receives a strongly negative KL reward (-2.76). By episode 2 the policy has learned "keeping tokens = negative reward" and drops nearly everything ($\rho = 0.005$). The fix—distributing total KL equally to all tokens (**??**)—eliminates the collapse: the same policy explores compression ratios from 2% to 75% over subsequent episodes.

Table 4. Drop-all collapse from per-token KL rewards (top) vs. stable training with shared KL distribution (bottom). Same policy and hyperparameters; only the KL distribution method differs.

Episode	ρ	KL reward	Sparse reward	Entropy
Per-token KL (broken)				
1	0.567	−2.757	0.011	0.637
2	0.005	−0.120	0.011	0.044
3	0.000	0.000	−0.500	0.029
Shared KL (fixed)				
1	0.567	−2.750	0.045	0.637
2	0.021	−0.330	0.101	0.045
3	0.005	−0.135	0.121	0.092
4	0.188	−1.938	0.072	0.438
5	0.697	−2.109	0.000	0.553

Pathology 2: Ground-truth F1 produces near-zero reward. Our initial sparse reward compared the compressed-prompt LLM output against ground-truth SQuAD answers: $\text{F1}(\mathcal{M}(\tilde{\mathbf{x}}), y^*)$. At $\rho \approx 0.5$, this yields F1 ≤ 0.03—the LLM almost never produces the exact extractive answer from a compressed prompt. After switching to faithfulness ($\text{F1}(\mathcal{M}(\tilde{\mathbf{x}}), \mathcal{M}(\mathbf{x}))$), the sparse reward increased to 0.05–0.12, a 5–10× improvement. This aligns with PCRL and TACO-RL, both of which use faithfulness rather than ground-truth comparison.

4.5. REINFORCE with Continuous Reward

We report results from an early REINFORCE training run with the DistilRoBERTa policy (82M params, pretrained encoder), Llama-3.2-1B frozen LLM (4-bit quantized), and continuous sparse reward $r = \text{faithfulness} \cdot (1 - \rho)$. The SCST baseline uses a greedy rollout, with k=4 sampled episodes per prompt, trained on 2000 SQuAD episodes.

Motivation. The gated reward from PCRL (reward $= 1 - \rho$ if faithfulness ≥ 0.9, else -0.01) produced zero SCST advantage with Llama-3.2-1B because faithfulness never exceeded 0.9 at any compression level. Continuous reward provides non-zero gradient signal for any non-zero faithfulness, enabling the policy to learn from partial success.

Table 5. REINFORCE + continuous reward training phases. Three distinct regimes emerge over 1000 episodes: initial exploration, conservative learning with occasional high-faithfulness episodes, and collapse from over-aggressive compression.

Phase	Entropy	Ratio	Faithfulness	Reward
Exploration (0–100)	0.69→0.65	0.43–0.55	0.0–0.17	0.0–0.08
Conservative (100–200)	0.65→0.54	0.70–0.82	spikes to 1.0	0.0–0.28
Collapse (800–1000)	0.54→0.65	0.23–0.38	0.0	0.0

Analysis. The training exhibits a three-phase pattern (**??**). In the initial exploration phase, entropy starts near maximum (0.69) as the keep-biased initialization produces near-uniform decisions. In the conservative learning phase, the policy learns to keep more tokens (ratio 0.70–0.82), entropy drops to 0.54 showing specialization, and occasional faithfulness=1.0 episodes produce the highest rewards (up to 0.28). In the collapse phase, the policy overcorrects—compression becomes too aggressive (ratio drops to 0.23–0.38), faithfulness collapses to 0.0 because the LLM cannot produce coherent output from such heavily compressed prompts, and entropy drifts back toward random as all reward signal vanishes. The core failure mode is that REINFORCE makes independent per-token decisions with no look-ahead: once the policy begins compressing too aggressively, the resulting zero faithfulness provides no gradient signal to correct course.

Transition to MCTS. These results demonstrate that continuous reward provides gradient signal where the gated reward produced none, but REINFORCE with per-token sampling still fails to converge. The policy oscillates because each token decision is made independently: the agent cannot anticipate whether its current compression level will produce coherent LLM output. Monte Carlo Tree Search (MCTS) addresses this by planning ahead over chunk-level decisions, evaluating complete compression masks before committing. By exploring the action space via tree search, MCTS can find compression masks that maintain faithfulness even when most random samples produce zero reward. This gives us a principled comparison: reactive per-token policy (REINFORCE) vs. planning-based compression (MCTS).

4.6. REINFORCE vs. MCTS with Multiple Reward Functions

We conduct a systematic comparison of REINFORCE and MCTS across four reward functions, using the DistilRoBERTa policy (82M params, frozen encoder with trainable MLP or self-attention head) and Llama-3.2-1B-Instruct (4-bit quantized) as the target LLM. All runs use k=8 samples, 128 episodes per optimizer step, and shared hyperparameters: lr $= 3\times10^{-5}$, entropy coefficient $\beta = 0.001$, max grad norm 5.0, quality threshold $\tau = 0.6$. Each run trains for 3000–5000 episodes on 500 SQuAD training examples with 20 validation examples for periodic evaluation. We report the best eval F1 (faithfulness-weighted task score) across all checkpoints.

Reward functions. We compare four reward formulations:

$$r_{\text{cont}} = f \cdot (1-\rho) \tag{7}$$

$$r_{\text{gate}} = \begin{cases} 1-\rho & \text{if } f \geq \tau \\ -p & \text{otherwise} \end{cases} \tag{8}$$

$$r_{\text{soft}} = \sigma(k(f-\tau)) \cdot (1-\rho) \tag{9}$$

$$r_{\text{harm}} = \frac{2f(1-\rho)}{f+(1-\rho)} \tag{10}$$

where $f = \text{ROUGE-L}(\mathcal{M}(\tilde{\mathbf{x}}), \mathcal{M}(\mathbf{x}))$ is faithfulness, τ=0.6, p=0.01, and k=10 (sigmoid steepness). Continuous reward provides smooth gradients but no quality floor. Gated reward (**?**) enforces a hard quality threshold but creates a cliff discontinuity. Soft-gated smooths the cliff via sigmoid. Harmonic mean balances faithfulness and compression symmetrically.

Policy heads. We compare two classification heads atop the frozen DistilRoBERTa encoder: (1) **MLP** ($\sim$197K params): two linear layers with GELU activation, scoring each token independently; (2) **Self-attention** ($\sim$460K params): 4-head self-attention layer followed by a linear projection, enabling cross-token importance scoring. Both use Xavier normal initialization (gain=0.1) on the output weight with bias $[0, 0.2]$; the initialization design is analyzed in **??**.

Table 6. 2×2×2 factorial: greedy eval on 20 validation samples after 2000 episodes. All runs use DistilRoBERTa + self-attention head with Xavier init (gain=0.1, bias $[0, 0.2]$), entropy coeff 0.01, accumulation steps 4, k=8 samples, max_new_tokens=16, trained on SQuAD 2.0 with Llama-3.2-1B-Instruct (4-bit). Uncompressed baseline F1=0.134. Faith. = faithfulness ROUGE-L.

Algorithm	Reward	LR	Faith.	F1 ↑	ρ
REINFORCE	Harmonic	3×10^{-4}	0.548	0.091	0.382
MCTS	Harmonic	3×10^{-4}	0.510	0.079	0.275
REINFORCE	Harmonic	1×10^{-4}	0.504	0.079	0.345
MCTS	Harmonic	1×10^{-4}	0.463	0.060	0.257
MCTS	Gated	3×10^{-4}	0.587	**0.128**	0.574
MCTS	Gated	1×10^{-4}	0.671	0.121	0.573
REINFORCE	Gated	1×10^{-4}	0.544	0.096	0.408
REINFORCE	Gated	3×10^{-4}	0.507	0.071	0.387
LLMLingua-2 (ρ=0.3)				0.037	0.300
Uncompressed baseline				0.134	1.000

Key findings. **??** reveals several patterns:

(1) Harmonic reward acts as an aggressive compression dial. Harmonic runs achieve 65–74% compression (ρ 0.26–0.38) but faithfulness drops to 0.46–0.55. The symmetric treatment of faithfulness and compression in the harmonic mean creates strong incentive to compress even when it damages output quality.

(2) Gated reward preserves task performance. MCTS+gated achieves 43% compression (ρ=0.57) while preserving 90–95% of uncompressed task F1 (0.121–0.128 vs. 0.134 baseline). The hard threshold at τ=0.6 acts as a quality floor that prevents over-compression.

(3) The reward function controls the compression–faithfulness tradeoff, not the algorithm. Across both REINFORCE and MCTS, harmonic consistently produces lower ratios and lower faithfulness than gated. Algorithm choice has a smaller effect than reward choice on all metrics.

(4) lr=3×10^{-4} causes entropy collapse on long runs. All four 3×10^{-4} runs ended with entropy 0.03–0.27 (near-deterministic), while all 1×10^{-4} runs maintained entropy 0.40–0.62. The higher learning rate finds good compression masks faster but collapses to a fixed policy that cannot generalize.

(5) MCTS+gated at 3×10^{-4} achieves the highest task F1 (0.128)—3.5× better than LLMLingua-2 at comparable compression—but with collapsed entropy (0.03), suggesting it memorized training prompts rather than learning a generalizable compression strategy.

(6) Gated reward is fragile with small LLMs. PCRL (**?**) reports success with gated reward at τ=0.9. We had to lower τ to 0.6 because Llama-3.2-1B cannot reliably achieve 0.9 faithfulness. Even at τ=0.6, the cliff creates training instability: identical compression masks yield inconsistent ROUGE-L scores across prompts due to the small LLM's noisy generation, making the threshold effectively random from the policy's perspective.

4.7. Scaling to Llama-3.1-8B

To test whether reward design findings transfer across LLM capability, we repeat the MCTS comparison using Llama-3.1-8B (4-bit NF4 quantized) as the target LLM while keeping the DistilRoBERTa policy and self-attention head unchanged. All 8B runs use lr $= 1\times10^{-5}$, entropy coefficient $\beta = 0.01$ (10× larger than the 1B runs to prevent premature entropy collapse with the noisier gated reward), and $\tau = 0.5$ for gated variants. We train for 3000 episodes on 500 SQuAD training examples. Note: the 8B training runs used `max_new_tokens=64` (vs. 16 for the 1B runs), producing more verbose LLM outputs during training. Final evaluation uses `max_new_tokens=16` for fair comparison with 1B results.

MCTS training stability. An initial MCTS run with shared policy/value optimizer exhibited gradient norm spikes to 24.3 (4.85× the clip threshold of 5.0), while a parallel REINFORCE baseline stayed in the 2–6× range. The root cause: the value head's MSE loss gradients (16K params, zero-initialized, predicting $\sim$0 against targets of

0.3–0.7) dominated the shared `clip_grad_norm_`, effectively starving the 460K-param policy head of learning signal. Three fixes were applied: (1) separate Adam optimizers with independent gradient clipping (policy: lr $= 1\times10^{-5}$, clip 5.0; value: lr $= 1\times10^{-3}$, clip 1.0); (2) Huber loss (`smooth_l1`, β=0.1) for the value head to cap gradient magnitude on outlier targets; (3) blend warmup annealing the learned value weight from 0 to 0.7 over 50 episodes so the zero-initialized value head does not corrupt early MCTS search.

Table 7. MCTS with Llama-3.1-8B (4-bit NF4). All use DistilRoBERTa + self-attention head + decoupled optimizers (policy lr $= 1\times10^{-5}$, value lr $= 1\times10^{-3}$). These runs used stochastic eval; greedy re-evaluation is pending. † = still training at time of writing.

Configuration	F1 ↑	ρ	Notes
MCTS+Gated (decoupled)	0.160	0.749	Peak at ep 1500, then drift
MCTS+Continuous (decoupled)†	0.129	0.720	Stable, no drift
REINFORCE+Gated	0.089	0.717	Conservative drift by ep 250

Key findings at 8B scale. (1) Gradient decoupling is essential for MCTS. An initial MCTS run with shared policy/value optimizer exhibited gradient norm spikes to $24\times$ the clip threshold, while a parallel REINFORCE baseline stayed at 2–6$\times$. After decoupling, MCTS+Gated reached 0.160 F1 — surpassing REINFORCE's 0.089 peak by 80%.

(2) Conservative drift persists at 8B scale. REINFORCE+Gated peaked at episode 200, then declined to 0.044 by episode 600 as the policy retreated to conservative compression (ρ rising from 0.67 to 0.73). MCTS+Gated also exhibited drift (0.160 $\rightarrow$ 0.039) but recovered to 0.136 — the tree search provides enough exploration to escape the conservative attractor that traps REINFORCE permanently.

(3) Continuous reward avoids drift. MCTS+Continuous maintained stable training without the oscillation seen in gated reward. Peak F1 is lower (0.129 vs. 0.160), consistent with the 1B finding that smooth rewards trade peak performance for stability.

(4) 8B enables more aggressive compression. At comparable task scores, the 8B policy compresses at $\rho \approx 0.72$ (dropping 28% of tokens) vs. $\rho \approx 0.77$ for 1B. The more capable LLM tolerates greater information loss.

4.8. Cross-LLM Transfer

The compression policy (frozen DistilRoBERTa encoder + trainable head) is LLM-agnostic: it decides which tokens to keep based on the prompt alone, without access to the target LLM's weights. This raises a natural question: do policies trained with one LLM transfer to another? We evaluate each checkpoint with both Llama-3.2-1B and Llama-3.1-8B using greedy policy actions and `max_new_tokens=16`.

Table 8. Cross-LLM transfer: greedy eval F1 for policies trained on one LLM and evaluated on both. "Matched" = eval LLM matches training LLM. "Transfer" = eval LLM differs from training.

		Eval on 1B		Eval on 8B	
Policy	**Reward**	F1	ρ	F1	ρ
Trained on Llama-3.2-1B					
MCTS+Attn	Harmonic	**TBD**	TBD	TBD	TBD
MCTS+Attn	Soft-gated	TBD	TBD	TBD	TBD
MCTS+Attn	Gated	TBD	TBD	TBD	TBD
MCTS+Attn	Continuous	TBD	TBD	TBD	TBD
REINFORCE	Continuous	TBD	TBD	TBD	TBD
Trained on Llama-3.1-8B					
MCTS+Attn	Gated (dec.)	TBD	TBD	**TBD**	TBD
MCTS+Attn	Continuous (dec.)	TBD	TBD	TBD	TBD

4.9. Initialization and Greedy Evaluation

The output layer initialization critically determines whether the policy can learn compression. With a zero-initialized output weight matrix W mapping hidden states h to logits $z = Wh + b$, the chain rule yields:

$$\frac{\partial \mathcal{L}}{\partial h} = \frac{\partial \mathcal{L}}{\partial z} \cdot W^{\top} = 0 \tag{11}$$

No gradient reaches the hidden layers, so only the output layer itself can update—and it does so slowly (weight norm grows by <2% over 5000 episodes). The policy appears to learn under stochastic evaluation because the bias prior $[0, b]$ produces random masks via softmax sampling, and random token dropping achieves nonzero faithfulness on SQuAD by chance. Greedy evaluation reveals the truth: the policy keeps >99% of tokens.

Two initialization choices interact to determine when greedy compression emerges. First, Xavier normal initialization (gain=0.1) on the output weight provides weight magnitudes ($\sim$0.03 per element, $\sim$0.44 total contribution) large enough to produce token-specific logit differences. Second, the bias gap between keep and drop classes controls the decision threshold: a gap of $[0, 0.2]$ requires weight contributions of only 0.2 to flip argmax, while a gap of $[0, 1.0]$ requires contributions of 1.0—25$\times$ harder to overcome.

?? shows the resulting training dynamics. With Xavier init and bias $[0, 0.2]$, the policy exhibits a phase transition: greedy compression ratio remains near 1.0 until $\sim$step 1000, then drops rapidly as learned weights cross the decision boundary for increasing numbers of tokens.

All results in **??** use Xavier initialization and greedy evaluation.

Table 9. Greedy compression ratio during training (MCTS+harmonic, lr=1×10^{-4}). A phase transition occurs between steps 1000–1500 as learned weights overcome the initialization bias gap.

Training Step	Greedy Compression Ratio
500	0.972
1000	0.959
1500	0.427
2000	0.252

4.10. Extended Training and Dense Reward

We extend the two best harmonic runs from the factorial (MCTS and REINFORCE at lr=1×10^{-4}) from 2000 to 4000 episodes. We also test a novel dense reward signal: *teacher-forced cross-entropy* (CE), which measures how surprised the compressed-prompt LLM is by the tokens the full-prompt LLM generated. Unlike input-space KL (which requires token alignment destroyed by the text bridge), output-space CE is naturally aligned since both forward passes use the same target token sequence:

$$r_{\text{CE}} = -\alpha \cdot \frac{1}{K} \sum_{i=1}^{K} \log P_{\mathcal{M}}(y_i \mid \tilde{\mathbf{x}}, y_{1:i-1}) \qquad (12)$$

where $y_{1:K}$ are the tokens generated by the LLM on the *full* prompt, and $P_{\mathcal{M}}(\cdot \mid \tilde{\mathbf{x}}, \ldots)$ is the LLM's distribution conditioned on the *compressed* prompt. This requires one additional forward pass per episode (teacher-forced, not autoregressive, so no extra generation cost).

?? reports greedy eval results. Extended training to 4000 episodes improves compression substantially: MCTS reaches ρ=0.447 (vs. 0.257 at step 2000) while maintaining 0.93 faithfulness, and REINFORCE pushes to ρ=0.223 (78% compression) though with collapsed entropy. The CE runs achieve more aggressive compression than the 2000-episode factorial runs at the same episode count: MCTS+CE reaches ρ=0.325 with 0.73 faithfulness.

Table 10. Extended training and dense CE reward on SQuAD (greedy eval, 20 validation samples). "Ext" runs continue harmonic training from step 2000 to 4000. "CE" runs add teacher-forced cross-entropy (α=0.1) alongside harmonic sparse reward. All use lr=1×10^{-4}, DistilRoBERTa + self-attention head. Faith. = faithfulness ROUGE-L, F1 = task F1 against ground truth, Ent. = final action entropy (max 0.693).

Configuration	Eps	Faith.	F1 ↑	ρ	Ent.
MCTS+Harm. (ext)	4000	0.517	0.090	0.447	0.40
REINFORCE+Harm. (ext)	4000	0.484	0.060	0.223	0.17
MCTS+Harm.+CE	2000	0.536	**0.096**	0.325	0.48
REINFORCE+Harm.+CE	2000	0.591	0.091	0.544	0.51
Uncompressed baseline				0.134	1.000
—					

MCTS+CE is the most efficient configuration: it matches the compression level of MCTS+harmonic at 4000 episodes (ρ=0.33 vs. 0.45) in half the training time (2000 episodes), with higher faithfulness (0.54 vs. 0.52) and better task F1 (0.096 vs. 0.090). The CE signal provides denser gradient information than ROUGE-L alone: while ROUGE-L treats "France" and "Germany" identically as non-matching tokens, CE captures that the compressed model assigned 40% probability to the correct answer—partial credit that helps the policy distinguish between good and bad compression masks. MCTS benefits more from CE than REINFORCE; the tree search can exploit the richer signal to find better masks, while REINFORCE+CE drifts conservative (ρ=0.54) as the CE penalty discourages aggressive exploration.

5. Analysis

We analyze the compression behavior of the best-performing policy qualitatively, examining which tokens the policy keeps and drops at 75% compression (MCTS+harmonic, ρ=0.257, 20 validation samples).

The policy learned a coherent token-selection strategy organized around three priorities. Question tokens and the instruction prefix are always retained—the policy never drops the actual question or the "Answer the following" framing. Entity names from the context passage are preserved (e.g., "Normans," "Nourmands," "Normandi," "Normanni"), as are structural tokens that anchor the passage's factual skeleton ("centuries," "descendants," "West Francia"). Articles, prepositions, filler words, and detailed elaboration sentences are dropped.

This strategy works well for simple factual questions where the answer is obvious from minimal context. For the question "What country is Normandy in?" the compressed prompt retains enough geographic context for the LLM to answer correctly, achieving faithfulness 1.0. However, it fails when the answer depends on specific details in the dropped context. For "From which countries did the Norse originate?" the policy drops the sentence listing Denmark, Iceland, and Norway—the entity names appear within elaboration that the policy classifies as filler. The LLM hallucinates an incorrect answer from the remaining context, yielding faithfulness 0.15.

This failure mode is systematic: at 75% compression, the policy must drop the majority of context tokens, and it has no mechanism to predict which details the LLM will need to answer a specific question. The policy learns *general* token importance (entities > function words) but not *question-conditional* importance (which entities matter for *this* question). The gated reward configurations, which compress less aggressively (ρ=0.57), avoid this failure by retaining enough context that most answer-bearing details survive.

6. Discussion

Reward design matters most. Across all algorithm and architecture combinations in our diagnostic sweep (**??**), the reward signal was the primary determinant of training stability. Dense KL reward prevents collapse by providing per-token gradient signal; sparse F1 alone yields zero terminal reward at aggressive compression ratios. The hybrid combination (KL-dense + sparse faithfulness) was the only reward type that produced nonzero task performance within 100 episodes.

Dense rewards require careful credit assignment. Per-token KL rewards are appealing in principle—they provide the dense gradient signal that PCRL and TACO-RL lack—but introduce the keep/drop asymmetry (**??**). Our shared distribution fix (**??**) resolves this while preserving the dense signal. This pathology is not specific to prompt compression: any binary selection problem with per-item rewards risks the same asymmetry when one action (select) incurs a cost and the other (skip) is free. The shared distribution approach generalizes to other RL formulations with asymmetric action costs.

Faithfulness vs. accuracy as reward. The switch from ground-truth F1 to faithfulness F1 produced a 5–10$\times$ increase in reward signal (**??**). This is not merely a scaling difference: faithfulness measures whether the LLM's *behavior is preserved*, which is the actual goal of compression. Ground-truth accuracy conflates compression quality with the LLM's inherent ability to answer correctly, introducing irreducible noise into the reward signal. PCRL and TACO-RL both use faithfulness, though neither explicitly discusses this design choice or contrasts it against ground-truth comparison.

Bandit vs. sequential. The contextual bandit—which makes independent per-token decisions with no temporal credit assignment—outperformed REINFORCE in the diagnostic sweep despite being a simpler algorithm. This suggests that for binary token classification, the sequential MDP structure may add variance without improving credit assignment. The bandit's simplicity also makes the keep/drop reward asymmetry easier to diagnose and fix.

Architecture: frozen encoder + lightweight head. DistilBERT exhibited drop-all collapse within 100 episodes when fully fine-tuned (entropy$\rightarrow$0.002, $\rho \rightarrow$0.004; **??**). Our solution—freezing the DistilRoBERTa encoder and training only a lightweight head (197K–460K params)—eliminates this failure mode while leveraging pretrained representations. Entropy remains above 0.46 across all 5000+ episode runs (**??**). The self-attention head variant ($\sim$460K params) substantially outperforms the MLP head ($\sim$197K params) on MCTS, suggesting that cross-token importance scoring—enabled by the attention mechanism—is valuable for compression decisions.

Gated reward and LLM capability. Our results reveal an interaction between reward function design and target LLM quality that prior work does not address. PCRL (**?**) uses gated reward ($1 - \rho$ if faithfulness ≥ 0.9, else -0.01) successfully with a capable LLM. With our smaller Llama-3.2-1B (4-bit quantized), we lowered τ from 0.9 to 0.6 and still observed reward drift. The root cause is that the cliff discontinuity amplifies noise in the faithfulness signal: with a small LLM, the same compression mask produces inconsistent ROUGE-L across different prompts, so the policy cannot reliably learn where the threshold lies. Continuous reward degrades gracefully under the same noise—partial faithfulness still produces partial reward.

Scaling from Llama-3.2-1B to Llama-3.1-8B (4-bit) reveals a second failure mode: *conservative drift*. With 8B, faithfulness is more reliable, so we raised τ from 0.6 to 0.5. REINFORCE initially achieves a 15–22% pass rate (episodes clearing the threshold), but pass rates decline to 5–7% by episode 250 as the policy gradually increases its compression ratio from ρ=0.67 to ρ=0.71. The mechanism is a perverse incentive in the SCST advantage structure: with a near-zero baseline (EMA ≈ 0.004), conservative episodes that barely pass the threshold (ρ=0.7, reward= 0.3) receive outsized positive advantage ($0.3 - 0.004 \approx 0.3$), strongly reinforcing "keep most tokens." Aggressive compression (ρ=0.5, potential reward= 0.5) fails the threshold $\sim$90% of the time, so its expected advantage is negative. The policy rationally retreats to conservative compression, but this narrow strategy generalizes poorly across prompts, causing the pass rate to decline even as the best-episode faithfulness remains high (≥ 0.85). This is not entropy collapse—entropy stays above 0.58—but rather reward-driven convergence to a local optimum where the policy compresses too little to be useful.

This finding suggests that gated reward is not fundamentally incompatible with REINFORCE (PCRL demonstrates otherwise), but is *fragile when the target LLM is small or noisy*—and even with capable LLMs, the reward cliff can drive conservative drift when the threshold is hard to pass consistently. Practitioners should match reward function sharpness to LLM capability: use gated thresholds only when the LLM can reliably achieve the threshold at the desired compression level, otherwise prefer smooth reward functions.

7. Limitations and Future Work

Single frozen LLM. All experiments use Llama-3.1-8B as the frozen target LLM. Compression policies may learn

token importance patterns specific to this model's attention structure. Whether policies trained on one LLM transfer to another (e.g., Mistral-7B, Gemma-2) is an open question. Prior work on soft prompt compression (**?**) similarly evaluates on a single model family.

English-only evaluation. Our datasets (SQuAD 2.0, MeetingBank) are English-only, and Llama-3.1's tokenizer is optimized for English. This is a significant limitation: tokenizer fertility varies dramatically across languages—from $\sim$2 tokens per word in English to $\sim$10 in agglutinative languages like Telugu (**?**). Higher fertility means each token carries less information, fundamentally changing the compression problem. Dropping one BPE token in Turkish may destroy an entire morpheme, while dropping one in English may only remove a subword fragment. Gutierrez-Vasques et al. (**?**) show that BPE compression exploits qualitatively different patterns across 47 languages—productive morpheme merges in agglutinative languages vs. idiosyncratic frequent-substring merges in analytic languages. Whether a single compression policy can handle this variation, or whether language-specific policies are needed, is entirely unstudied.

The one encouraging signal comes from LLMLingua-2 (**?**), whose English-trained compressor (using a multilingual XLM-RoBERTa backbone) transfers to Chinese without explicit multilingual training. An immediate extension of our work would be evaluating the best English-trained policy on XQuAD (**?**), a cross-lingual SQuAD variant covering 11 languages, to measure how compression quality degrades as a function of tokenizer fertility and morphological typology.

Hard vs. soft compression. We study *hard* prompt compression (discrete token keep/drop), but an active parallel line of work compresses prompts into continuous embeddings. Gist tokens (**?**) achieve 26$\times$ compression by learning special token activations, ICAE (**?**) compresses 512 tokens into 32–128 memory slots via a LoRA-adapted encoder, and 500xCompressor (**?**) reaches up to 480$\times$ compression into KV-cache values. These soft methods may generalize better across languages since they operate in continuous embedding space rather than on discrete tokens. A direct comparison of hard token-dropping (our approach) vs. soft embedding compression at matched compression ratios would clarify when each paradigm is preferable.

Tokenizer-compression interaction. Recent work suggests that tokenizer design interacts non-trivially with compressibility. Schmidt et al. (**?**) find that raw compression ratio correlates only weakly ($r = 0.241$) with downstream performance across 64 language models—pre-tokenization design matters more. MrT5 (**?**) demonstrates that byte-level models learn language-specific compression rates, achieving 75–80% sequence reduction. Morphologically-aware tokenizers like MorphBPE (**?**) produce tokens with more consistent semantic content by respecting morpheme boundaries, which could make token-level compression decisions more reliable. Investigating whether morpheme-boundary constraints improve RL-based compression is a natural extension.

Computational overhead. Token-level RL compression requires running the frozen LLM for reward computation during training. Our hybrid reward needs three forward passes per episode: one for full-prompt logits (cached for KL), one for compressed-prompt generation, and one for original-prompt generation (faithfulness baseline). Whether the inference savings from compression justify this training cost depends on deployment volume—a question our work does not address quantitatively.

8. Conclusion

We study reward design for RL-based token-level prompt compression, making four contributions.

First, a $2\times2\times2$ factorial over algorithm (REINFORCE, MCTS), reward (harmonic, gated τ=0.6), and learning rate (1×10^{-4}, 3×10^{-4}) reveals the reward function as the primary lever for the compression–faithfulness tradeoff. Harmonic reward and gated reward define a Pareto frontier: harmonic achieves up to 74% compression (ρ=0.26) at the cost of reduced faithfulness, while gated reward preserves 90–95% of uncompressed task F1 at 43% compression (ρ=0.57). With extended training (4000 episodes), MCTS+harmonic achieves 55% compression (ρ=0.45) with 93% faithfulness, the best balance of compression and quality.

Second, we propose *teacher-forced cross-entropy* as a dense reward signal for prompt compression. Unlike input-space KL divergence (which requires token alignment between encoder and LLM vocabularies), output-space CE compares the LLM's predictive distributions on a shared target sequence, providing per-token gradient signal without architectural constraints. MCTS+CE achieves ρ=0.33 in 2000 episodes—comparable to MCTS+harmonic at 4000 episodes—suggesting the denser signal accelerates learning.

Third, we identify and resolve two reward pathologies—a keep/drop asymmetry in dense KL rewards that drives drop-all collapse, and near-zero signal from ground-truth comparison—extending prior work (PCRL, TACO-RL) with dense reward signals these methods lack.

Fourth, qualitative analysis reveals that the policy learns interpretable compression strategies: retaining question tokens and entity names while dropping function words and elaboration. This succeeds on simple factual queries but fails when dropped context contains the answer, ex-

posing a fundamental limitation of general (non-question-conditional) token importance scoring at high compression ratios.

Higher learning rates (3×10^{-4}) cause entropy collapse across all configurations, producing near-deterministic policies. Freezing the encoder and training only a lightweight self-attention head ($\sim$460K params) eliminates the RL-induced entropy collapse observed with full fine-tuning of pretrained encoders.

References

Anonymous. MorphBPE: A morpho-aware tokenizer bridging linguistic complexity for efficient LLM training across morphologies. *arXiv preprint arXiv:2502.00894*, 2025a.

Anonymous. The token tax: Systematic bias in multilingual tokenization. *arXiv preprint arXiv:2509.05486*, 2025b.

Artetxe, M., Ruder, S., and Yogatama, D. On the cross-lingual transferability of monolingual representations. In *Proceedings of ACL*, 2020.

Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Ge, T., Hu, J., Wang, X., Chen, S.-Q., and Wei, F. In-context autoencoder for context compression in a large language model. *arXiv preprint arXiv:2307.06945*, 2023.

Gutierrez-Vasques, X., Bentz, C., and Samardžić, T. Languages through the looking glass of BPE compression. *Computational Linguistics*, 49(4):943–1001, 2023.

Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y., and Qiu, L. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of EMNLP*, 2023.

Jung, H. and Kim, D. PCRL: Prompt compression with reinforcement learning. *IEEE Access*, 2024.

Kallini, J. et al. MrT5: Dynamic token merging for efficient byte-level language models. In *Proceedings of ICLR*, 2025.

Li, Y., Dong, B., Guerin, F., and Lin, C. Compressing context to enhance inference efficiency of large language models. In *Proceedings of EMNLP*, 2023.

Li, Z. et al. 500xcompressor: Generalized prompt compression for large language models. *arXiv preprint arXiv:2408.03094*, 2024.

Mu, J., Li, X. L., and Goodman, N. Learning to compress prompts with gist tokens. *arXiv preprint arXiv:2304.08467*, 2023.

Pan, Z., Wu, Q., Jiang, H., Xia, M., Luo, X., Zhang, J., Lin, Q., Ruhle, V., Yang, Y., Lin, C.-Y., et al. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Proceedings of ACL*, 2024.

Rajpurkar, P., Jia, R., and Liang, P. Know What You Don't Know: Unanswerable questions for SQuAD. In *Proceedings of ACL*, 2018.

Sanh, V., Debut, L., Chaumond, J., and Wolf, T. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

Schmidt, L., Madhyastha, P. S., et al. Tokenization is more than compression. In *Proceedings of EMNLP*, 2024.

Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

Shandilya, S. et al. TACO-RL: Task aware prompt compression optimization with reinforcement learning. In *Proceedings of ACL*, 2025.